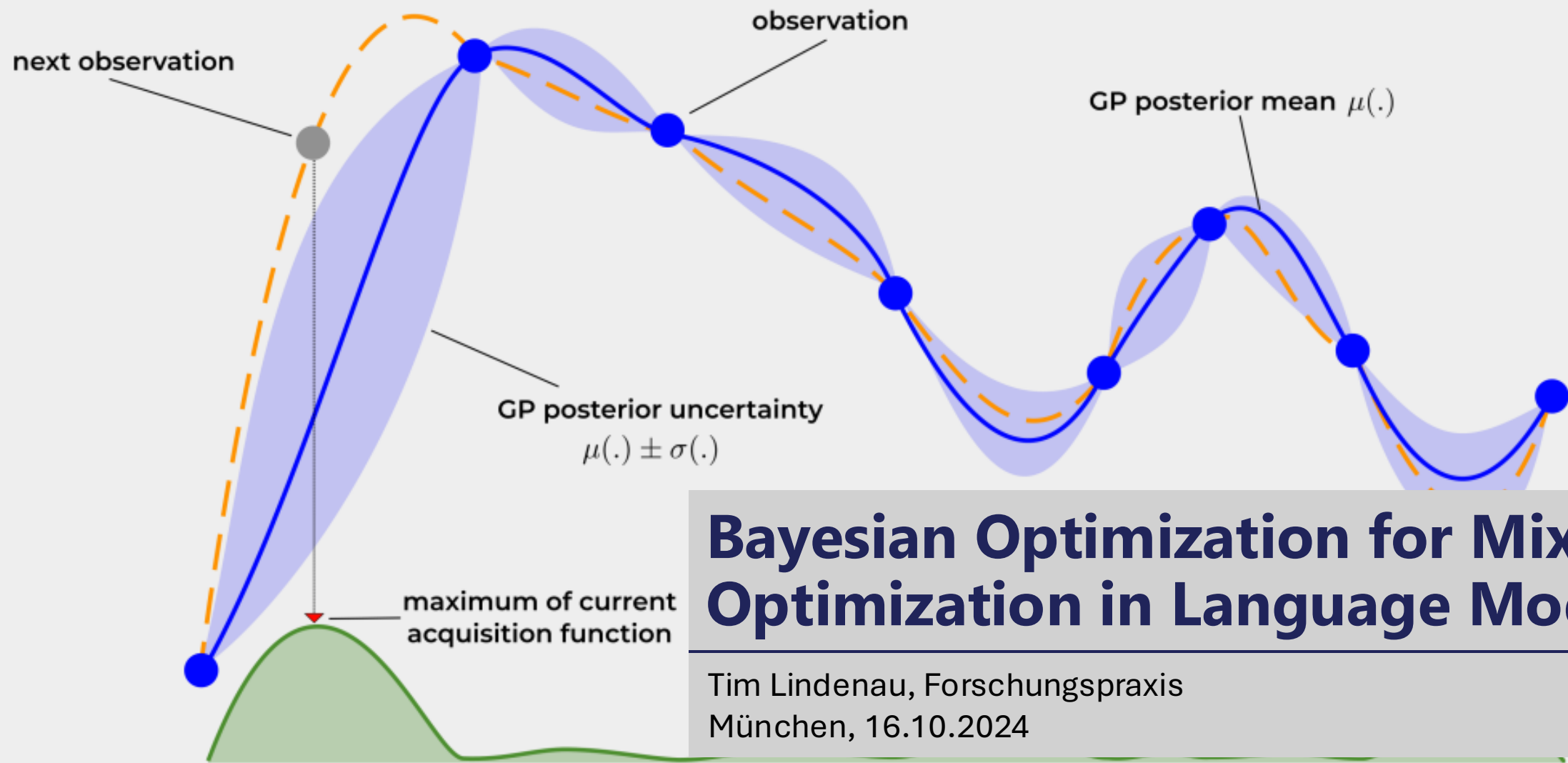




Bayesian Optimization for Mixture Optimization in Language Models?

Tim Lindenau, Forschungspraxis
München, 16.10.2024



Motivation

Data Mixing Important for Achieving Highest Model Performance on Constrained Compute Budget

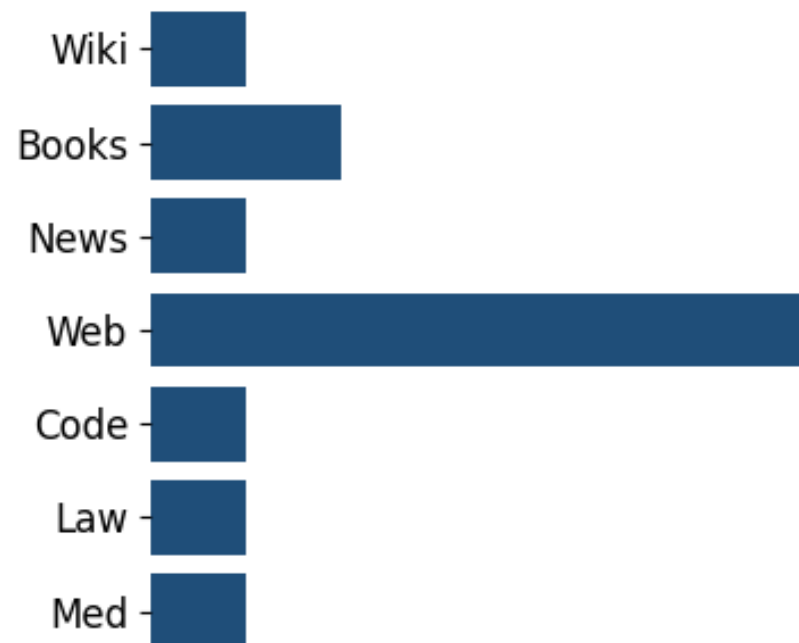
Problem Setup – Data Mixing:

- LLMs pre-trained with data from k **domains** (Wikipedia, Web, Code, etc.)
- **Not** all domains of **same quality**
- For a **fixed budget**, how much of **each domain** to train on

Biggest Challenge – Mixture Optimization without significant compute overhead:

- **Identify optimal mixture** on **small proxy model**, assuming **scale invariance**
- Focus on **sample efficiency** requiring as **few proxy model trainings** as possible

Mixing proportions of different datasets for pre-training



Existing Mixture Optimization Approaches and their Pros & Cons

Name	Method	Pros & Cons
Online Mixing	• Data Mixing while training the production grade model • Multi-Armed Bandit to model which domain to sample next token • Domain perplexity as reward function	+ Negligible run-time overhead without pre-training proxy models - Missing interpretability into overall optimal mixture
DoReMi	• Robust optimization minimizing worst case performance • Reference model is pre-trained on uniform mixture • Optimal mixture to minimize per-domain excess loss compared to reference model	+ Only two proxy models required + Optimization criterion promises good generalizability - Identified mixture very sensitive to model architecture
Mixing Laws	• Formulate parametrized law to predict performance as function of mixture weights • Law fitted to data based on some number of proxy evaluations • Optimal mixture predicted as minimizer of fitted law	+ Complexity reduction of search space - Risk of modelling errors in law, (example: BiMix not modelling any domain interactions)

Three key challenges to tackle for novel mixture optimization approach

Existing Mixture Optimization Approaches and their Pros & Cons

Name	Method	Pros & Cons
Online Mixing	• Data Mixing while training the production grade model • Multi-Armed Bandit to model which domain to sample next token • Domain perplexity as reward function	+ Negligible run-time overhead without pre-training proxy models - Missing interpretability into overall optimal mixture
DoReMi	• Robust optimization minimizing worst case performance • Reference model is pre-trained on some mixture • Optimal mixture to minimize per-domain excess loss compared to reference	+ Only two proxy models required + Optimization criterion promises good generalizability - Identified mixture very sensitive to model architecture
Mixing Laws	• Formulate parametrized law to model performance as function of mixture weights • Law-fitting based on limited number of evaluations • Optimal mixture predicted as minimizer of law	+ Complexity reduction of search space bounded too law - Risk of modelling errors in law, (example: BiMix not modelling any domain interactions)

1

Compute Efficiency: Minimal overhead unlocked by high sample efficiency

2

Flexibility: Reduced dependency on pre-defined functional shapes to prevent modelling errors

3

Insight: Identified mixture must provide insight into the true quality/utility of each domain after training



Method: Bayesian Optimization for Data Mixing

BO is a Sequential Optimization Process Towards Function Optimization

- All **previous mixture weights** x_i and **performance value** y_i collected in dataset

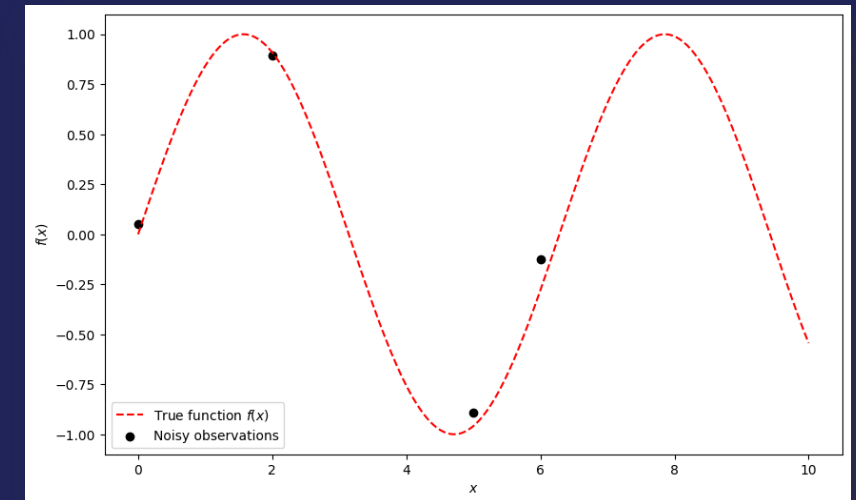
$$D_t = \{(y_i, x_i)\}_{i=0}^t$$

- **Gaussian Process** provides **probability distribution** of performance at each sampling point

$$g(x, y | D) = \Pr(f(x) = y | D)$$

- **Acquisition function** measures utility of next-weight x_{t+1} , trading off **exploration** and **exploitation**
- **Maximizing acquisition** function gives **next mixture weight** in smart way
- **Optimization loop repeated** for multiple rounds

BO Step 1:



Noisy Sampling Locations of
1D Sine collected

BO is a Sequential Optimization Process Towards Finding Function Optimums

- All **previous mixture weights** x_i and **performance value** y_i collected in dataset

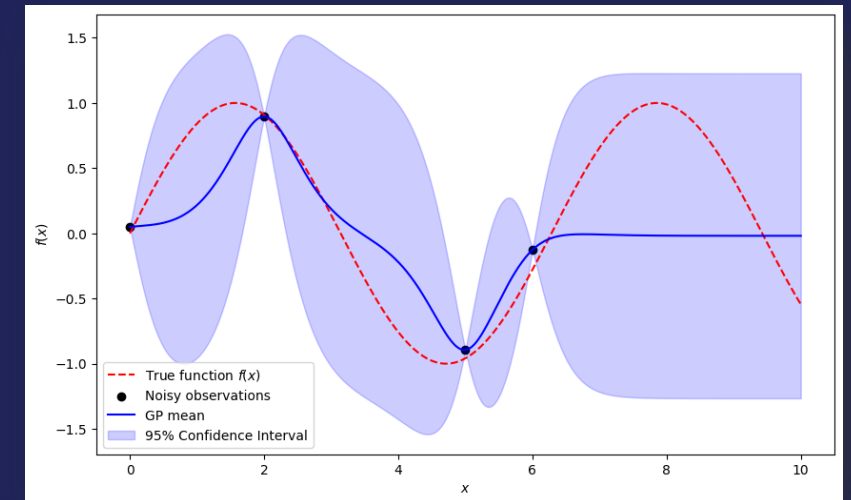
$$D_t = \{(y_i, x_i)\}_{i=0}^t$$

- **Gaussian Process** provides **probability distribution** of performance at each sampling point

$$g(x, y | D) = \Pr(f(x) = y | D)$$

- **Acquisition function** measures utility of next-weight x_{t+1} , trading off **exploration** and **exploitation**
- **Maximizing acquisition** function gives **next mixture weight** in smart way
- **Optimization loop repeated** for multiple rounds

BO Step 2:



Gaussian Process fitted to previously collected **dataset**

BO is a Sequential Optimization Process Towards Finding Function Optimums

- All **previous mixture weights** x_i and **performance value** y_i collected in dataset

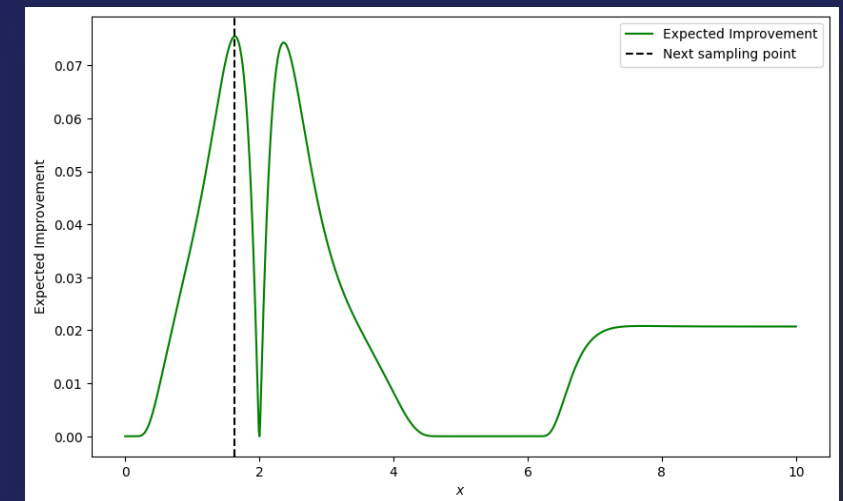
$$D_t = \{(y_i, x_i)\}_{i=0}^t$$

- **Gaussian Process** provides **probability distribution** of performance at each sampling point

$$g(x, y | D) = \Pr(f(x) = y | D)$$

- **Acquisition function** measures utility of next-weight x_{t+1} , trading off **exploration** and **exploitation**
- **Maximizing acquisition** function gives **next mixture weight** in smart way
- **Optimization loop repeated** for multiple rounds

BO Step 3:



Maximizing acquisition function provides
next **sampling location** to test

BO is a Sequential Optimization Process Towards Finding Function Optimums

- All **previous mixture weights** x_i and **performance value** y_i collected in dataset

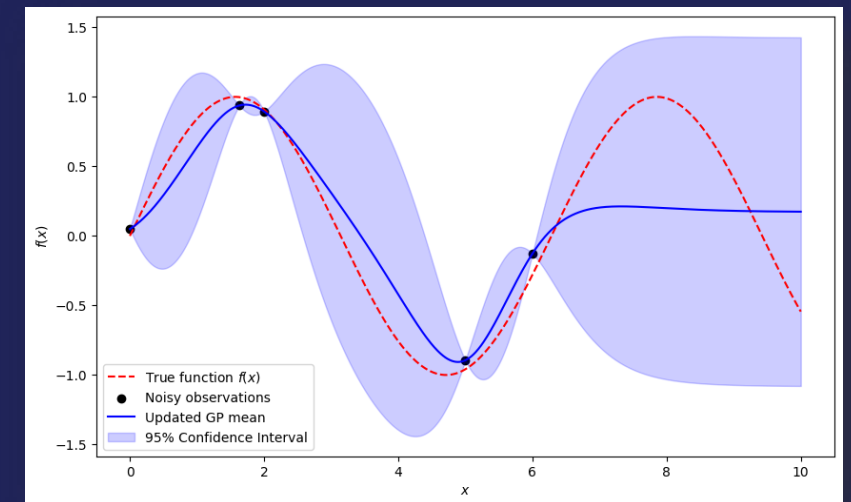
$$D_t = \{(y_i, x_i)\}_{i=0}^t$$

- **Gaussian Process** provides **probability distribution** of performance at each sampling point

$$g(x, y | D) = \Pr(f(x) = y | D)$$

- **Acquisition function** measures utility of next-weight x_{t+1} , trading off **exploration** and **exploitation**
- **Maximizing acquisition** function gives **next mixture weight** in smart way
- **Optimization loop repeated** until stopping criterion

BO Step 4:



Optimization loop repeated with **updated knowledge**

BO is a Sequential Optimization Process Towards Finding Function Optimums

- All **previous mixture weights** x_i and **performance value** y_i collected in dataset

$$D_t = \{(y_i, x_i)\}_{i=0}^t$$

- **Gaussian Process** provides **probability distribution** of performance at each sampling point

$$g(x, y | D) = \Pr(f(x) = y | D)$$

- **Acquisition function** measures utility of next-weight x_{t+1} , trading off **exploration** and **exploitation**
- **Maximizing acquisition** function gives **next mixture weight** in smart way
- **Optimization loop repeated** until stopping criterion

BO has potential for
high-sample efficiency,
without **systematic**
errors introduced by
mixing laws



Experiments

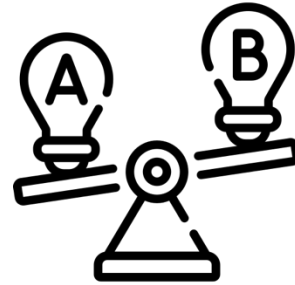
Experimental Setup

Data Sources



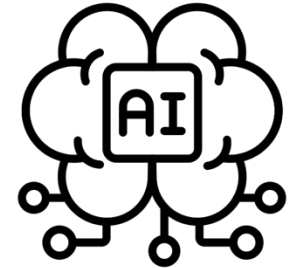
1. **Books:** Book Texts
2. **Common Crawl:** Web Data
3. **The Stack:** Code Data
4. **Pes20:** Academic Papers
5. **Reddit:** Social Media

Baselines



1. **Random:** Random Search
2. **Uniform:** 20% uniform weights
3. **LLaMa:** Weights reported
4. **BiMix:** SOTA Mixing Law
5. **MixLaws:** SOTA Mixing Law

Model & Metric



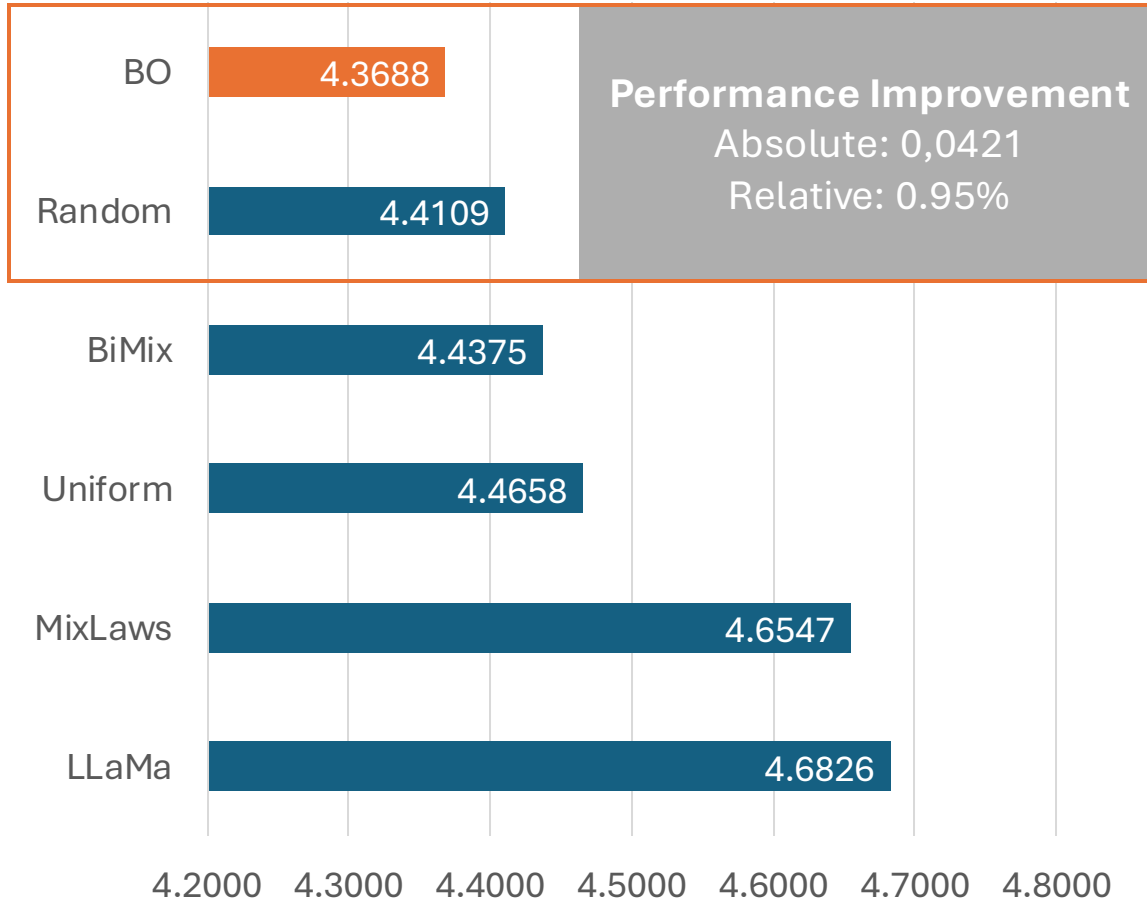
- **25M** parameters model size, trained **Chinchilla optimal**
- **64 trials** for each approach
- **Average perplexity** across all domains is **optimization criterion¹**

¹Each domain validated on held-out set, average compute across domains

BO is Effective and Outperforms All Other Baselines

Performance comparison against benchmarks

[Values in avg. log perplexity]

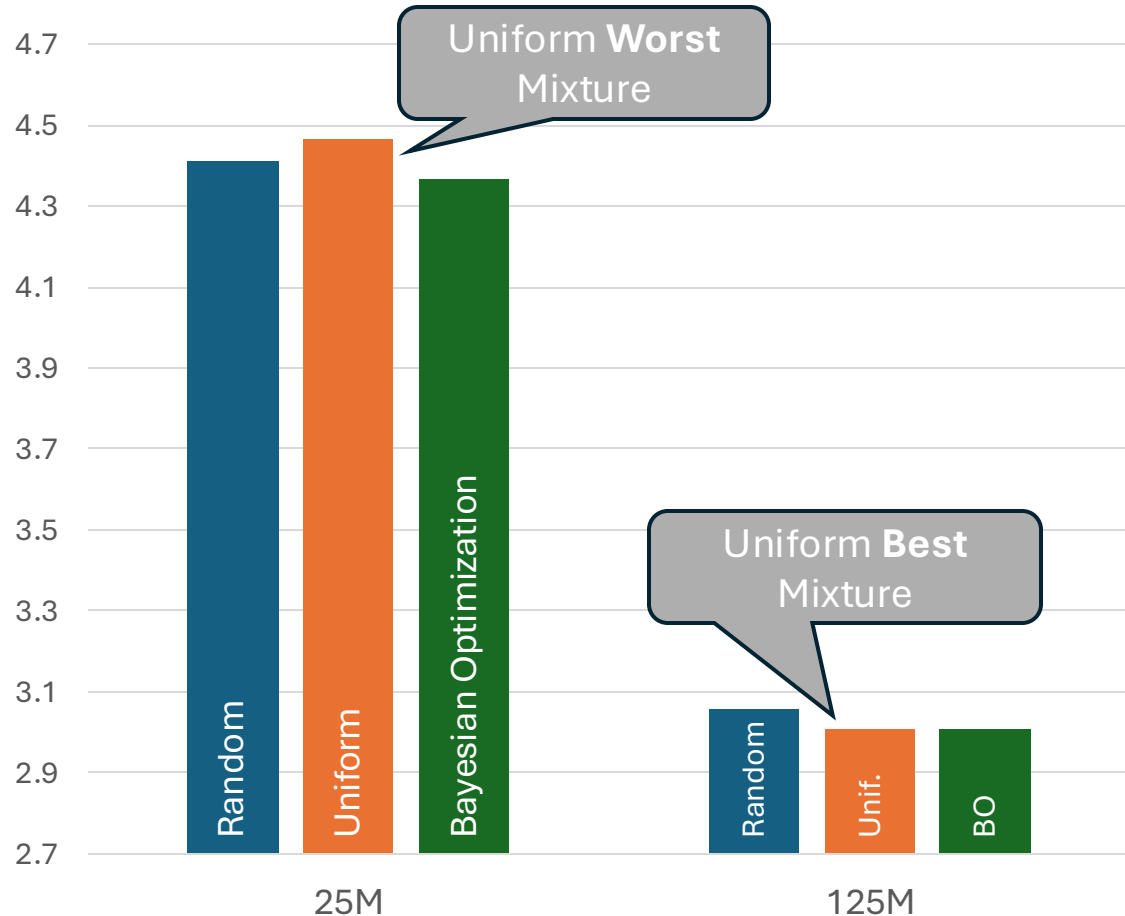


- **BO effective** and **identifies best-performing** mixture
- **$\Delta 0.04$ improvement significant** and larger than experiment variance of $\sigma < 0.02$
- **No other** approach capable to **outperform random search**
- **MixLaws** with major **struggle** to model **true function shape** leading to bad performance
- **LLaMa weights worst**. High focus on CC could be disadvantage in large model¹

¹According to Kang et al.

Missing Scale Invariance Hinders Real World Application of BO

Scaling up mixture from 25M model to 125M model [Values in avg. log perplexity]

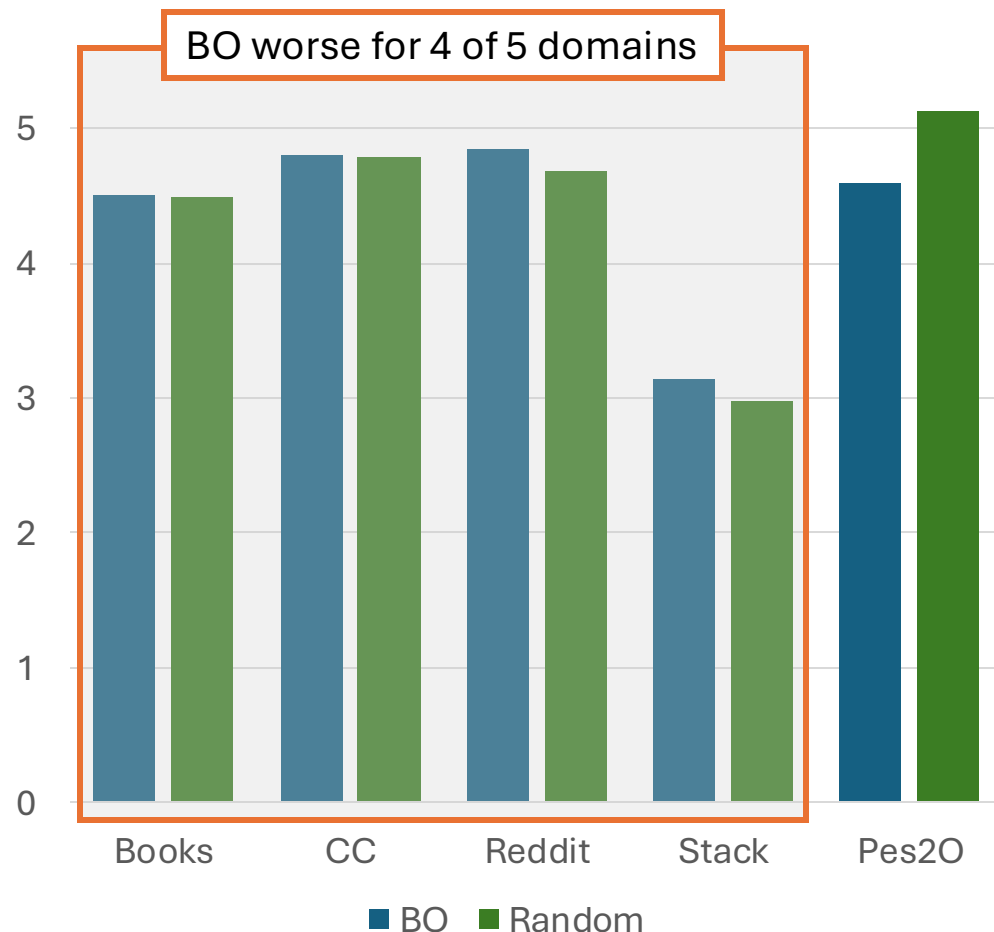


- **Assumption: Optimal mixture** identified at small proxy model **translates** to **larger** production **model**
- **Experiment:** When scaling up **uniform mixture** switches **from worst to best** performing mixture
- Utility of given **mixture** directly **depends** on **model size**

➡ **Limited usability of our method to optimize mixture of large models right now**

Avg. Perplexity Criterion can Create Undesirable Optimization Incentives

Best BO mixture compared to best Sobol mixture per domain [Values in avg. log perplexity]



Goal of Data Selection:

- Create model that **performs well on all training domains**
- Additional **generalization** to novel domains **desirable**

Limitations of Avg. Perplexity Criterion:

- Risk that model **trades of domains** to minimize overall loss (expert model)
- **Experiment: Random search better** than BO **in four of five domains** despite worse average performance



Conclusion

Conclusion: BO Effective at Identifying Optimal Mixtures at Proxy Model Scale but Limitations currently Hinder Real-World Usage

Result 1:

BO effective for optimizing mixtures of one architecture

- **BO only approach** capable of **outperforming** random search
- **Sample efficiency** expected from theory **validated** in practice

Result 2:

Identified limitations prevent adaptation of method in practice

- **Scale invariance** assumption **invalidated**. **Translating** insights from **proxy to production** model is **challenging** in practice
- **Limitations** of **average performance criterion** identified. Generalizing well **requires different** optimization **objective**
- **Not just our approach** suffers from limitations
→ **More research** required for **principled data mixing**